

Recommendation: BidReady AI

Tagline: Turn complex RFPs into evidence-backed, submission-ready response plans.

Build **BidReady AI**, an AI workspace for small agencies, IT consultancies, and startups responding to client RFPs (Requests for Proposals). Users upload an RFP and their company documents—capability deck, case studies, certifications, past proposals—and receive:

- A requirement-by-requirement compliance matrix
- A risk and missing-evidence report
- Cited draft answers grounded only in uploaded evidence
- A collaborative review and export workflow

This is the strongest option because it is more than “upload a PDF and summarize it.” It is a practical, workflow-driven SaaS product with visible AI reasoning, document processing, streaming, storage, authentication, cloud architecture, and excellent UX potential.

The project directly meets every mandatory guideline: responsive frontend, secure FastAPI backend, LLM integration, progressive streaming, Docker, public HTTPS AWS deployment, and environment-based secrets. It also gives you substantial material for the prompt-engineering and architecture portions of the report.

Why this beats common student projects

Common project	Why evaluators see it often	Why BidReady AI is stronger
Generic chatbot	Usually one prompt and a chat box	Has a real business workflow, citations, review states, and measurable outcomes
PDF summarizer	Limited to extraction and summary	Converts documents into action: requirements, gaps, ownership, drafts, and readiness score
AI tutor	Common and difficult to differentiate	Targets a real B2B pain point with clear SaaS potential
Content generator	Often produces unverified text	Produces controlled, evidence-grounded content with source citations
Task planner	Good UX but weaker technical depth	Combines document ingestion, retrieval, streaming, structured AI output, collaboration, and cloud services

Its key advantage is explainability. During evaluation, you can demonstrate a clear journey:

```
flowchart LR
    A["Upload RFP + company evidence"] --> B["Extract and index content"]
    B --> C["AI finds requirements"]
    C --> D["AI maps evidence and gaps"]
    D --> E["Team reviews cited response drafts"]
    E --> F["Export response plan"]
```

That story is easy to understand, professionally relevant, and substantially more impressive than a chat interface.

Product definition

Project name

BidReady AI

Problem statement

Small consulting firms and startups often lose time and miss important requirements while responding to lengthy RFPs. Information is scattered across proposal documents, case studies, and certifications. Manual review is slow, error-prone, and makes it hard to verify whether every answer is supported by evidence.

Target users

- IT service agencies
- Cloud and software consultancies
- Startup founders bidding for enterprise work
- Sales and proposal teams
- Freelance teams responding to client briefs

Real-world use cases

- An agency uploads a 40-page RFP and its portfolio before preparing a proposal.
- A startup identifies security, pricing, and compliance requirements before deciding whether to bid.
- A proposal manager assigns RFP sections to teammates and tracks completion.
- A consultancy drafts evidence-based answers for technical capability questions.
- A founder gets a “bid readiness” score before spending days on a proposal.

USP

BidReady AI does not merely summarize documents—it creates a traceable proposal-readiness workspace where every AI-generated recommendation and draft answer links back to supporting evidence or explicitly identifies the missing evidence.

Features

Core features — must have

1. Secure sign-up and login

- Email/password authentication
- User-specific workspaces and projects

2. Create an RFP project

- Project name, client name, deadline, bid value range, team members

3. Document upload

- Upload RFP PDF and supporting company documents
- Store originals securely in Amazon S3
- Show ingestion status

4. AI requirement extraction

- Extract requirements into structured categories:
 - Technical
 - Commercial
 - Compliance
 - Security
 - Timeline
 - Submission requirements

5. Compliance matrix

- Requirement
- Priority
- Supporting evidence
- Gap status
- Owner
- Draft status
- Confidence score

6. Evidence-grounded response drafting

- Generate draft answers using only uploaded material
- Include citations such as “Source: Case Study A, page 3”
- Clearly say “insufficient evidence” when required

7. Streaming AI responses

- Drafts visibly appear token by token
- Meet the guideline’s progressive-response requirement

8. Bid readiness dashboard

- Completion percentage
- High-risk gaps
- Unassigned requirements
- Requirements without evidence
- Upcoming deadline

9. Responsive mobile and desktop UI

- Mobile-friendly dashboard and review workflow

10. Export

- Export compliance matrix as CSV
- Export proposal response draft as DOCX or PDF if feasible; Markdown/HTML export is acceptable for the first version

Advanced features — good to have

- Semantic search across uploaded evidence
- Requirement assignment to team members

- Comments and approval/reject workflow
- Version history for generated drafts
- AI-generated executive summary of bid risks
- “Go / No-Go” recommendation with transparent reasons
- Requirement filters by risk, category, owner, and status
- Manual evidence selection before generating a response
- Document page preview alongside a cited draft
- Email notification for assigned high-priority requirements

Stretch features — only if core work is stable

- Automatic RFP deadline extraction
- Multi-language RFP support
- Proposal scoring rubric simulator
- Branded client-ready export
- “Similar past answer” suggestions from prior projects
- Voice walkthrough of bid risks
- Multi-tenant organization billing mockup
- Admin analytics dashboard

Do not build stretch features until the core workflow is deployed, tested, and documented.

Complete system architecture

```

flowchart TB
    U["User browser"] --> FE["Next.js frontend<br/>AWS App Runner"]
    FE -->|"HTTPS + JWT"| API["FastAPI backend<br/>AWS App Runner"]

    API --> AUTH["Amazon Cognito"]
    API --> DB["Amazon RDS PostgreSQL<br/>+ pgvector"]
    API --> S3["Amazon S3<br/>original uploads"]
    API --> LLM["Amazon Bedrock<br/>Claude model"]
    API --> EMB["Embedding model<br/>via Bedrock"]
    API --> SM["AWS Secrets Manager"]
    API --> CW["Amazon CloudWatch"]

    S3 --> INGEST["Document ingestion worker"]
    INGEST --> DB
    EMB --> DB

    API -->|"Server-Sent Events"| FE
  
```

Frontend

- **Next.js + TypeScript**
- Tailwind CSS and shadcn/ui
- Responsive pages for landing, authentication, workspace, project dashboard, document upload, requirement review, and response editor
- Fetch API with Server-Sent Events for streaming output

Backend

- **FastAPI + Python**
- REST API for projects, documents, requirements, drafts, comments, and exports
- Background document-processing job flow
- Pydantic validation
- SQLAlchemy ORM and Alembic migrations
- FastAPI `StreamingResponse` for AI response streaming

Database

- Amazon RDS PostgreSQL
- Enable `pgvector` for semantic document retrieval
- Store users, organizations, projects, parsed sections, requirement records, evidence links, drafts, comments, and audit logs

LLM integration

Preferred: **Amazon Bedrock with a Claude model.**

Why:

- Strong AWS story for the evaluation

- Secure IAM-based access
- No LLM API key exposed to browser code
- Streaming support
- High-quality reasoning and structured outputs

Use an embedding model to convert document chunks into vectors for retrieval. Before drafting an answer, retrieve only relevant chunks, then instruct the LLM to cite those chunks.

Authentication

Preferred: **Amazon Cognito**.

- Hosted or custom login UI
- JWT tokens
- Backend validates JWTs
- Users can only access their own organization and projects

A faster alternative is Clerk or Supabase Auth, but Cognito better supports the AWS architecture narrative.

File storage

- Amazon S3 private bucket
- Presigned upload URLs
- S3 object keys separated by organization/project/document
- Backend downloads/processes documents; browser never gets broad bucket access

Docker

Use two Dockerfiles:

- frontend/Dockerfile
- backend/Dockerfile

Use `docker-compose.yml` locally for frontend, API, and PostgreSQL.

AWS services

Service	Purpose
AWS App Runner	Deploy frontend and backend containers with public HTTPS
Amazon RDS PostgreSQL	Persistent relational data and pgvector
Amazon S3	Secure document storage
Amazon Bedrock	LLM and embeddings
Amazon Cognito	Authentication and JWT identity
AWS Secrets Manager	Production secrets
AWS CloudWatch	Logs, errors, and basic monitoring
AWS IAM	Least-privilege service permissions
AWS Budgets	Cost alert required by the guidelines

Deployment pipeline

```
flowchart LR
    A["Feature branch"] --> B["Pull request"]
    B --> C["Automated checks"]
    C --> D["Merge to main"]
    D --> E["GitHub Actions builds Docker image"]
    E --> F["Amazon ECR"]
    F --> G["AWS App Runner deployment"]
    G --> H["CloudWatch health checks"]
```

If CI/CD setup is too time-consuming, deploy manually from ECR first, then add GitHub Actions before submission.

Security best practices

- Never store secrets in source code or frontend variables.
- Use `.env.example` ; commit no real credentials.
- Store production secrets in AWS Secrets Manager.
- Use IAM roles for App Runner to access S3 and Bedrock.
- Keep S3 bucket private; use short-lived presigned URLs.
- Validate file type, size, and content before processing.
- Add rate limiting to AI-generation routes.
- Enforce organization/project ownership on every backend query.
- Sanitize document content before placing it into prompts.
- Log request metadata, never raw credentials or complete sensitive documents.
- Configure AWS Budget alerts from Day 1.

Streaming implementation

1. User clicks “Generate Draft.”
2. Frontend opens an SSE connection to `POST /api/v1/drafts/stream`.
3. Backend retrieves relevant evidence chunks.
4. Backend invokes Bedrock streaming.
5. Each token/chunk is forwarded as an SSE event.
6. Frontend appends content live in the editor.
7. Backend saves the completed draft and citations after the stream finishes.

Recommended tech stack

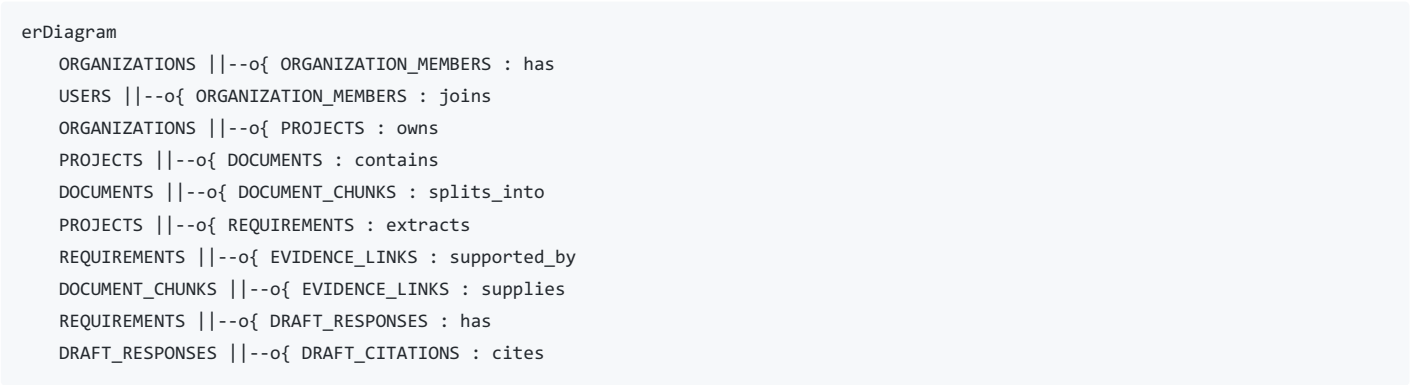
Layer	Choice	Why
Frontend	Next.js, TypeScript	Modern full-stack frontend, excellent routing and responsive UI support
UI	Tailwind CSS, shadcn/ui	Professional UI quickly without looking like a default template
Backend	FastAPI, Python	Fast development, ideal for LLM/document workflows, typed request validation
Database	PostgreSQL + pgvector	Reliable relational data plus semantic retrieval
ORM	SQLAlchemy	Mature Python database tooling
Migration	Alembic	Professional schema migration history
LLM	Amazon Bedrock Claude	Strong quality and AWS-native architecture
Storage	Amazon S3	Durable, scalable document storage
Authentication	Amazon Cognito	AWS-native JWT authentication
Deployment	Docker + App Runner	Straightforward container deployment and public HTTPS
CI/CD	GitHub Actions + ECR	Clear industry-standard deployment pipeline
Observability	CloudWatch	Deployment logs and runtime debugging
Testing	Pytest, Playwright	Backend, API, and critical UI-flow testing

Database schema

Table	Key fields
users	id, cognito_sub, email, name, created_at
organizations	id, name, created_by, created_at
organization_members	id, organization_id, user_id, role

projects Table	id, organization_id, name, client_name, deadline, status, created_by Key fields
documents	id, project_id, name, document_type, s3_key, page_count, processing_status, uploaded_by
document_chunks	id, document_id, page_number, content, embedding, metadata_json
requirements	id, project_id, category, title, description, priority, source_document_id, source_page, status, owner_id
evidence_links	id, requirement_id, document_chunk_id, relevance_score, notes
draft_responses	id, requirement_id, content, status, model_name, prompt_version, created_by
draft_citations	id, draft_response_id, document_chunk_id, citation_label
comments	id, project_id, requirement_id, draft_response_id, author_id, content, created_at
audit_logs	id, organization_id, user_id, action, entity_type, entity_id, metadata_json, created_at

Important relationships:



REST API structure

Base URL: /api/v1

Method	Endpoint	Purpose
POST	/auth/session	Exchange/validate auth session if needed
GET	/me	Get current user
GET	/organizations	List user organizations
POST	/organizations	Create organization
GET	/projects	List projects
POST	/projects	Create project
GET	/projects/{projectId}	Project dashboard data
PATCH	/projects/{projectId}	Update project
POST	/projects/{projectId}/members	Invite/assign teammate
POST	/projects/{projectId}/documents/upload-url	Get presigned S3 upload URL
POST	/projects/{projectId}/documents/{documentId}/process	Start ingestion
GET	/projects/{projectId}/documents	List project documents
GET	/projects/{projectId}/requirements	List/filter requirements

Method	Endpoint	Purpose
POST	/projects/{projectId}/requirements/extract	AI extraction from RFP
PATCH	/requirements/{requirementId}	Edit status, priority, owner
GET	/requirements/{requirementId}/evidence	Show linked evidence
POST	/requirements/{requirementId}/evidence/search	Find relevant company evidence
POST	/requirements/{requirementId}/drafts/stream	Stream grounded AI draft
GET	/requirements/{requirementId}/drafts	Draft history
PATCH	/drafts/{draftId}	Save reviewed draft
POST	/drafts/{draftId}/approve	Approve response
POST	/projects/{projectId}/comments	Create a comment
GET	/projects/{projectId}/export/compliance.csv	Export compliance matrix
GET	/projects/{projectId}/export/response.md	Export proposal draft
GET	/health	Deployment health check

Folder structure

```
bidready-ai/
├─ README.md
├─ docker-compose.yml
├─ .env.example
├─ docs/
│   └─ architecture.md
│   └─ prompting-strategy.md
│   └─ api-contract.md
│   └─ screenshots/
│   └─ evaluation-evidence.md
├─ frontend/
│   └─ Dockerfile
│   └─ src/
│       └─ app/
│           └─ page.tsx
│           └─ login/
│           └─ dashboard/
│           └─ projects/[projectId]/
│           └─ settings/
│       └─ components/
│           └─ dashboard/
│           └─ requirements/
│           └─ documents/
│           └─ ui/
│       └─ lib/
│       └─ types/
│       └─ tests/
├─ backend/
│   └─ Dockerfile
│   └─ app/
│       └─ main.py
│       └─ api/v1/
│       └─ core/
│       └─ db/
│       └─ models/
│       └─ schemas/
│       └─ services/
│           └─ bedrock_service.py
│           └─ ingestion_service.py
│           └─ retrieval_service.py
│           └─ export_service.py
│       └─ workers/
│   └─ alembic/
│   └─ tests/
└─ infra/
    └─ app-runner/
    └─ iam/
    └─ github-actions/
```

LLM prompt strategy

The guidelines allocate 20% to prompt engineering and documentation. Treat prompts as versioned product assets, not hidden implementation details. Save each prompt in `docs/prompting-strategy.md`, include intent, variables, expected format, test cases, failure cases, and refinements.

1. Requirement extraction prompt

You are an expert proposal compliance analyst.

Analyze the RFP text below and extract only explicit requirements.
Return valid JSON matching the supplied schema.

For each requirement include:

- title
- category: technical, commercial, compliance, security, timeline, submission
- priority: critical, high, medium, low
- requirement_text
- source_page
- acceptance_criteria
- clarification_needed: true or false

Rules:

- Do not invent requirements.
- Keep each requirement atomic and testable.
- Preserve the exact meaning of the RFP.
- Mark ambiguous text as clarification_needed=true.

RFP content:

{{rfp_chunks}}

2. Evidence matching prompt

You are matching a client requirement against internal company evidence.

Requirement:

{{requirement}}

Available evidence:

{{retrieved_chunks}}

Return JSON with:

- coverage_status: fully_covered, partially_covered, not_covered
- confidence: 0 to 100
- evidence_items: source name, page, short justification
- missing_information
- recommended_action

Rules:

- Use only the supplied evidence.
- Never claim compliance without evidence.
- If evidence is weak, choose partially_covered or not_covered.

3. Grounded response drafting prompt

You are a senior proposal writer. Draft a concise, credible answer to the client requirement.

Client requirement:

{{requirement}}

Approved evidence:

{{evidence_chunks}}

Instructions:

- Use only the evidence supplied.
- Do not make unsupported promises, metrics, certifications, or commitments.
- Write in professional proposal language.
- Include inline citations in this format: [Source name, p. X].
- If evidence is insufficient, begin with: "Evidence gap identified:" and state what is needed.
- Keep the answer below {{word_limit}} words.

4. Go/no-go risk summary prompt

Assess whether this organization should pursue the opportunity.

Use the requirement coverage data below. Produce:

1. Recommendation: Go, Conditional Go, or No-Go
2. Readiness score out of 100
3. Top five risks
4. Missing evidence or capabilities
5. Actions required before submission

Do not make commercial or legal decisions on behalf of the user.

State that this is an advisory assessment based on uploaded documents.

Prompt quality checklist

- Use structured JSON where the system needs structured data.
- Use low temperature for extraction and evidence matching.
- Use citations and “do not invent” constraints.
- Add clear output schemas.
- Test prompts with clean, ambiguous, incomplete, and contradictory documents.
- Keep a prompt-change log with before/after outputs and your reason for refinement.

Professional UI/UX flow

flowchart LR

```
A["Landing page"] --> B["Sign up / log in"]
B --> C["Workspace dashboard"]
C --> D["Create project"]
D --> E["Upload RFP + evidence"]
E --> F["AI processing screen"]
F --> G["Compliance dashboard"]
G --> H["Requirement detail"]
H --> I["Evidence panel + streaming draft editor"]
I --> J["Review, approve, export"]
```

Key screens

1. Landing page

- Clear headline: “Build stronger RFP responses with evidence, not guesswork.”
- Product workflow illustration
- Security and AWS deployment credibility
- Call to action: “Create your first bid workspace”

2. Workspace dashboard

- Active projects
- Upcoming deadlines
- Readiness scores
- “Create new project” action

3. Project setup

- Client, RFP title, deadline, team
- Drag-and-drop RFP upload
- Upload company evidence

4. Processing screen

- Friendly step indicator:
 - Uploading
 - Reading documents
 - Extracting requirements
 - Matching evidence
 - Preparing workspace

5. Compliance dashboard

- Readiness score card
- Requirement counts by status
- High-risk gaps
- Filterable requirement table

6. Requirement detail drawer/page
 - Requirement on the left
 - Linked evidence and page references on the right
 - Streaming response editor below
 - Approve, edit, regenerate, assign, and comment actions
7. Export page
 - Download compliance CSV
 - Download proposal response document
 - Submission checklist

Use calm B2B styling: navy/indigo, white space, status colors, readable tables, and strong empty states. Avoid a generic “AI glowing purple chatbot” aesthetic.

Team plan: three balanced roles

Use a **21-working-day plan**. If your internship has fewer days, keep core features and shorten advanced work; do not sacrifice deployment, streaming, documentation, or testing.

Member	Responsibilities	Expected deliverables	Main branches
Member 1: Product/UI engineer	UX, Next.js, dashboard, upload flow, streaming UI, responsive design	Complete responsive frontend, UI tests, screenshots	feature/ui-foundation , feature/project-dashboard , feature/streaming-editor
Member 2: Backend/AI engineer	FastAPI, database, ingestion, RAG, Bedrock prompts, SSE streaming	API, schema migrations, LLM pipeline, API tests, prompt documentation	feature/api-foundation , feature/rag-pipeline , feature/requirement-extraction
Member 3: Cloud/quality engineer	Docker, AWS, Cognito, S3, App Runner, CI/CD, monitoring, testing and report evidence	Deployment, Docker Compose, CI workflow, AWS diagram, test plan, report support	feature/docker-local , feature/aws-deployment , feature/ci-quality

All three must understand the full product because the guideline says each team member must independently explain every technical and design decision.

Daily work allocation

Days	Member 1	Member 2	Member 3
1–2	Wireframes, design system	API/schema design, prompt experiments	AWS account, budget alert, repo/CI setup
3–5	Landing, auth, dashboard shell	FastAPI, PostgreSQL models, migrations	Docker Compose, Cognito and S3 configuration
6–8	Upload and processing UI	S3 upload flow, PDF text extraction	Deploy initial frontend/backend containers
9–11	Requirement table/detail UX	Requirement extraction and evidence matching	Logs, health checks, staging deployment validation
12–14	Streaming editor, comments	Streaming endpoint, draft storage, citations	CI tests, security and environment configuration
15–16	Mobile polish, accessibility	Export API, filters, error handling	Production deployment, performance checks
17–18	Demo flow, screenshots	Prompt refinements, test fixtures	End-to-end tests, AWS documentation
19–21	UI fixes, demo rehearsal	API fixes, technical rehearsal	Final deployment, report assembly, submission checklist

Git workflow

- Protect `main`.
- Create short-lived feature branches from `main`.
- Use branch names such as:
 - `feature/project-dashboard`

- feature/rfp-ingestion
 - feature/aws-app-runner
 - fix/streaming-reconnect
- Every feature uses a pull request.
 - At least one teammate reviews each pull request.
 - PR description must include:
 - What changed
 - Screenshots or API example
 - Test evidence
 - Any environment/deployment impact
 - Use **squash merge** into `main` for clean history.
 - Never commit `.env`, API keys, Cognito secrets, or AWS credentials.
 - Tag milestone-ready versions:
 - `v0.1-foundation`
 - `v0.2-core-workflow`
 - `v1.0-submission`

Day-by-day roadmap

Day	Outcome
1	Read brief together, finalize scope, create concept note outline, create AWS budget alert
2	UX wireframes, architecture diagram, database/API design, repository setup
3	Initialize frontend/backend, Docker Compose, linting, branch rules
4	Authentication setup and initial UI
5	PostgreSQL schema, migrations, project CRUD
6	S3 private bucket and document-upload flow
7	PDF text extraction and ingestion-state UI
8	Save chunks and embeddings in pgvector
9	Build and test requirement-extraction prompt
10	Display requirements, categories, priorities, and filters
11	Build evidence matching and coverage status
12	Build cited draft-generation prompt
13	Implement FastAPI SSE streaming
14	Build frontend streaming editor and draft saving
15	Add dashboard readiness score and risk summaries
16	Add comments, assignment, and CSV export
17	Deploy container images to ECR/App Runner
18	Connect production S3, RDS, Cognito, Bedrock, and secrets
19	Mobile testing, security review, error states, rate limits
20	Complete report, prompt log, architecture diagrams, screenshots
21	Final live test, team rehearsal, submission verification

Milestones

1. **M1: Architecture approved** — Day 2
2. **M2: Local authenticated app working** — Day 5
3. **M3: Upload and document ingestion working** — Day 8
4. **M4: Requirement and evidence workflow working** — Day 11
5. **M5: Streaming cited response generation working** — Day 14
6. **M6: Complete user workflow and export ready** — Day 16
7. **M7: Public AWS HTTPS deployment live** — Day 18
8. **M8: Report, tests, screenshots, and final rehearsal complete** — Day 21

Risks and solutions

Challenge	Solution
PDF text extraction is poor	Start with text PDFs; add OCR only if time allows; display processing failure clearly
LLM invents unsupported claims	Retrieval-only context, explicit grounding prompt, citations, and “evidence gap” fallback
Bedrock model access is delayed	Request model access immediately; keep OpenAI API integration as documented fallback
Streaming is difficult	Implement a simple SSE endpoint first, then add reconnection/error handling
AWS costs increase	Budget alerts, small RDS instance/free tier, document limits, low-cost model, delete test resources
Team merge conflicts	Clear component ownership, small PRs, daily sync, frequent rebasing
Scope becomes too large	Freeze core scope by Day 8; advanced features only after M5
Evaluators cannot understand AI output	Show source citations, evidence coverage, readiness score, and demo with a prepared RFP

Evaluation-criteria mapping

Evaluation criterion	Weight	Evidence in BidReady AI	Why it scores well
Technical implementation and Vibe Coding methodology	25%	Full-stack app, document pipeline, database, streaming, tests, Docker, documented AI-assisted development	Clearly exceeds a basic frontend-plus-chat demo
Prompt engineering quality and documentation	20%	Versioned extraction, retrieval, evidence-matching, and drafting prompts; test cases and refinements	Prompting is central to product quality, not an afterthought
Cloud deployment and AWS architecture	20%	App Runner, RDS, S3, Cognito, Bedrock, IAM, Secrets Manager, CloudWatch, Budget alerts	Demonstrates meaningful AWS architecture and secure deployment
Application design and user experience	20%	Guided upload-to-compliance workflow, responsive dashboard, streaming editor, citations, clear status signals	UX is purpose-built around a real user workflow
Report quality, reflection, clarity	15%	Architecture, journey, challenges, prompt log, screenshots, metrics, lessons learned	The product produces an excellent narrative for the required report

Final execution checklist

Before development

- Finalize the scope as the core feature set only.
- Create a one-page concept note with title, problem, user, LLM, features, and expected outcome.
- Set up AWS Budget alerts.
- Create repository, protected `main`, issue board, and branch conventions.
- Create `docs/prompting-strategy.md` from Day 1.

During development

- Keep all keys in environment variables.
- Make every major AI feature show visible user value.
- Test with at least three sample RFP scenarios:
 - Well-supported proposal
 - Missing-evidence proposal
 - Ambiguous/complex proposal
- Take screenshots throughout, not only at the end.
- Record prompt changes and explain why each refinement improved output.
- Keep the public deployment updated regularly.

Before submission

- Verify the AWS URL opens in an incognito browser.
- Test desktop and mobile layouts.
- Confirm streaming works visibly.
- Confirm no secrets exist in frontend code or Git history.
- Confirm Dockerfiles build successfully.
- Confirm all uploaded documents remain private.
- Put the live AWS URL in both the **Concept Note** and **Project Report**, exactly as required.
- Include architecture, tech stack, prompting strategy, sample prompts, development phases, challenges, resolutions, learning reflection, and screenshots in the report.
- Rehearse a five-minute demo:
 1. Create project
 2. Upload RFP and evidence
 3. Show extracted requirements
 4. Open an evidence gap
 5. Generate a cited streaming draft
 6. Show readiness score
 7. Show AWS URL and architecture

BidReady AI is ambitious enough to feel like a genuine startup product, yet tightly scoped enough for three people to deliver if the team protects the core workflow and treats deployment/documentation as first-class deliverables..

I'd lock the scope to:

1. Login
2. Create RFP project
3. Upload RFP and company documents
4. Extract requirements with AI
5. Match each requirement with supporting evidence
6. Generate cited, streaming response drafts
7. Readiness dashboard showing covered vs. missing requirements
8. Dockerized public AWS deployment
9. Strong prompting and development documentation

Defer these until the MVP is live and polished:

- Organization/team management
- Comments and version history
- Assignments
- Semantic search UI
- Go/no-Go analysis
- Multiple export formats
- CI/CD automation
- Advanced Cognito configuration

One refinement: keep authentication simple. Use a managed provider only if the team already knows it; otherwise implement straightforward email/password login securely in the backend for the demo. The important requirement is secure server-side secrets and a working application—not using every AWS service.

A better delivery sequence is:

```
flowchart LR
    A["Foundation<br/>Login, projects, database"] --> B["Document workflow<br/>Upload and processing"]
    B --> C["AI workflow<br/>Requirements, evidence, cited drafts"]
    C --> D["Polish<br/>Dashboard, streaming, responsive UX"]
    D --> E["Ship<br/>Docker, AWS, report, demo"]
    E --> F["Only if time remains<br/>Advanced features"]
```

This keeps BidReady AI original and technically strong while making completion realistic for a three-person internship team.